

INFORME TÉCNICO

Resampleo y Análisis de Señales de Audio

Manipulación de Frecuencia de Muestreo y Cuantización

Lenguaje: Python 3

Fecha: Junio 2026

1. Descripción General

Este programa implementa un flujo de análisis y manipulación de señales de audio en Python. Trabaja sobre dos archivos de audio: una señal ya resampleada en formato WAV y una señal original en formato MP3. A partir de esta última, el programa aplica dos tipos de transformaciones: modificación de la velocidad de reproducción mediante cambio de frecuencia de muestreo, y reducción de la calidad mediante cuantización a 8 bits.

El flujo general del programa es el siguiente:

- Lectura e inspección de una señal de audio resampleada (WAV).
- Lectura de la señal de audio original (MP3).
- Conversión a mono si el audio es estéreo.
- Modificación de la velocidad de reproducción mediante factor de velocidad.
- Reducción de la calidad a 8 bits mediante cuantización uniforme.
- Reproducción interactiva de cada señal en un entorno Jupyter Notebook.

2. Librerías Utilizadas

Librería	Módulo / Alias	Propósito
IPython.display	Audio, display	Reproducción de audio interactivo en Jupyter.
Matplotlib	matplotlib.pyplot	Visualización de la forma de onda del audio.
NumPy	numpy (np)	Operaciones matemáticas sobre arrays de muestras.
Librosa	librosa, librosa.display	Análisis y procesamiento de señales de audio.
SoundFile	soundfile (sf)	Lectura y escritura de archivos de audio (WAV, MP3).

Nota: si bien librosa y librosa.display son importadas, en el código actual no se utilizan directamente. Su inclusión sugiere que el programa fue pensado para posibles extensiones, como el cálculo de espectrogramas o análisis de frecuencia.

3. Análisis de la Señal Resampleada

3.1 Lectura del Archivo WAV

El programa lee el archivo output_16bit.wav usando soundfile e imprime información descriptiva de la señal:

```
audio, sr = sf.read('output_16bit.wav')
print('Vector de la Señal Segmentada', audio)
print('Muestra del medio:', audio[50000:50010])
print('Largo array:', len(audio))
print('Frecuencia de Muestreo:', sr)
print('Duración:', len(audio)/sr)
print('Máximo valor:', np.max(np.abs(audio)))
```

3.2 Métricas Inspeccionadas

Dato	Código	Descripción
Vector de muestras	audio	Array NumPy con todos los valores de amplitud.
Muestra del medio	audio[50000:50010]	Ventana de 10 muestras en la posición 50000.
Largo del array	len(audio)	Número total de muestras en el archivo.
Frecuencia de muestreo	sr	Muestras por segundo del archivo WAV.
Duración (s)	len(audio)/sr	Duración total en segundos del audio.
Máximo valor	np.max(np.abs(audio))	Amplitud pico de la señal (debería ser ≤ 1.0).

3.3 Visualización de la Forma de Onda

La forma de onda completa se grafica con Matplotlib, permitiendo observar la distribución temporal de la amplitud de la señal resampleada:

```
plt.plot(audio)
plt.show()
```

4. Análisis de la Señal Original

4.1 Lectura del Archivo MP3

```
audio_original, sr_original = sf.read('AnálisisTextos.mp3')
print('Frecuencia de Muestreo original:', sr_original)
print('Duración original:', len(audio_original)/sr_original)
```

El archivo de origen es un MP3 llamado AnalisisTextos.mp3. soundfile lo lee y devuelve las muestras normalizadas en punto flotante (rango [-1.0, 1.0]) junto con la frecuencia de muestreo original.

4.2 Reproducción en Jupyter

La señal original se reproduce directamente en el notebook mediante el widget de audio de IPython:

```
IPython.display.display(Audio(audio_original, rate=sr_original))
```

4.3 Conversión a Mono

Si el archivo es estéreo (dos canales), se conserva únicamente el canal izquierdo (columna 0) para simplificar el procesamiento posterior:

```
if len(audio_original.shape) > 1:
    audio_original = audio_original[:, 0]
```

Un audio estéreo tiene forma (N, 2), mientras que uno mono tiene forma (N,). La condición detecta la presencia de una segunda dimensión para realizar la conversión.

5. Modificación de Velocidad por Resampleo

5.1 Concepto

Una forma sencilla de modificar la velocidad de reproducción de un audio sin alterar las muestras es cambiar la frecuencia de muestreo declarada al reproductor. Si la frecuencia se reduce a la mitad, el reproductor interpreta las muestras como si fueran de una señal más lenta, alargando la duración y bajando el tono (pitch).

$sr_modificado = sr_original \times factor_velocidad$

5.2 Implementación

```
factor_velocidad = 0.5
sr_modificado = int(sr_original * factor_velocidad)

print(f'Reproducir Modificado en Tiempo ({factor_velocidad}x):')
IPython.display.display(Audio(audio_original, rate=sr_modificado))
```

5.3 Efecto Resultante

Parámetro	Valor Original	Valor Modificado
Factor de velocidad	—	0.5x
Frecuencia de muestreo	sr_original (ej: 44100 Hz)	sr_original / 2 (ej: 22050 Hz)
Duración del audio	D segundos	D × 2 segundos
Tono (pitch)	Normal	Una octava más grave

6. Reducción de Calidad a 8 Bits

6.1 Concepto de Cuantización

La cuantización es el proceso de reducir la precisión de las muestras de audio limitando la cantidad de niveles de amplitud disponibles. Un audio de 16 bits tiene 65.536 niveles; uno de 8 bits tiene solo 256 niveles. Esto introduce un error de cuantización (ruido) audible, especialmente en señales de baja amplitud.

6.2 Implementación

```
niveles = 2 ** 8    # = 256

# Mapear de [-1, 1] a [0, 255]
audio_baja_calidad = np.round((audio_original + 1) * (niveles - 1) / 2)

# Volver a mapear a [-1, 1]
audio_baja_calidad = (audio_baja_calidad / (niveles - 1)) * 2 - 1

print('Reproducir con Baja Calidad (8 bits):')
IPython.display.display(Audio(audio_baja_calidad, rate=sr_original))
```

6.3 Etapas del Proceso

Etapa	Operación	Descripción
Escalado positivo	$(\text{audio} + 1) \times 255 / 2$	Mapea [-1, 1] al rango entero [0, 255].
Redondeo	np.round(...)	Reduce a 256 niveles discretos (8 bits).
Rescalado	$(\text{val} / 255) \times 2 - 1$	Devuelve el rango a [-1, 1] para reproducción.

El audio resultante mantiene la misma duración y frecuencia de muestreo que el original, pero presenta mayor ruido de cuantización al tener menos niveles de precisión.

7. Flujo Completo de Ejecución

Paso	Acción	Salida / Resultado
1	Leer output_16bit.wav	Array de muestras WAV + frecuencia de muestreo.
2	Imprimir métricas de la señal WAV	Largo, duración, máximo valor en consola.
3	Graficar forma de onda	Gráfico de amplitud vs. muestra.
4	Leer AnalisisTextos.mp3	Array de muestras MP3 + frecuencia de muestreo.

Paso	Acción	Salida / Resultado
5	Reproducir audio original	Widget de audio en Jupyter.
6	Convertir a mono (si necesario)	Array unidimensional de muestras.
7	Calcular <code>sr_modificado</code> (0.5x)	Frecuencia reducida a la mitad.
8	Cuantizar a 8 bits	Array con 256 niveles de amplitud.
9	Reproducir audio lento (0.5x)	Widget de audio en Jupyter (doble duración).
10	Reproducir audio 8 bits	Widget de audio en Jupyter (con ruido de cuantización).

8. Conceptos Clave de Procesamiento de Audio

Concepto	Definición
Frecuencia de muestreo (Hz)	Número de muestras tomadas por segundo. CD: 44100 Hz, Teléfono: 8000 Hz.
Cuantización (bits)	Precisión de cada muestra. 16 bits = 65536 niveles; 8 bits = 256 niveles.
Resampleo	Proceso de cambiar la frecuencia de muestreo de una señal.
Ruido de cuantización	Distorsión introducida al reducir niveles de amplitud.
Mono / Estéreo	Mono: un canal. Estéreo: dos canales (izquierdo y derecho).
Normalización [-1, 1]	Representación estándar de amplitud de audio en punto flotante.

9. Posibles Mejoras y Extensiones

- Agregar visualización del espectrograma con `librosa.display.specshow` para comparar las señales en el dominio frecuencial.
- Implementar resampleo de alta calidad con `librosa.resample` en lugar de modificar la tasa declarada al reproductor.
- Exportar las señales transformadas a archivos WAV con `sf.write` para uso externo al notebook.
- Calcular la SNR (relación señal-ruido) entre la señal original y la cuantizada a 8 bits.
- Permitir al usuario ingresar el factor de velocidad y la profundidad de bits de forma interactiva con widgets de Jupyter (ipywidgets).

- Añadir soporte para múltiples archivos de audio y procesar en lote.

10. Conclusión

El programa ilustra de manera práctica dos conceptos fundamentales del procesamiento digital de audio: el resampleo como herramienta para modificar la velocidad de reproducción, y la cuantización como modelo de reducción de calidad.

La implementación es concisa y directamente ejecutable en un entorno Jupyter Notebook, lo que la hace adecuada para fines didácticos en cursos de procesamiento de señales o audio digital. La estructura del código es clara y extensible, pudiendo incorporar fácilmente análisis espectrales, filtros digitales o exportación de resultados.